PhishDesk: Revised 8-Month Plan (v2)

Two equal owners · Person 1: Cybersecurity and backend · Person 2: AI/ML and dashboard · Status: planned, not completed

This revision keeps the structure of the original plan and fixes nine issues found in review: workload balance, missing open-source work, overloaded months, safeguards that arrived too late, no buffer, weak evaluation protocol, no tie-break rule, thin academic planning, and several small gaps.

1. What changed and why

#	Change	Reason
1	React dashboard moves to Person 2; Person 1 publishes the API spec and mock API early	Person 1 carried backend, three connectors, sandbox, remediation and deployment; Person 2 was lighter
2	A Community track (about 2-3 hours per week each from Month 2) runs beside the technical work	Contributors and stars need a README, demo, good first issues, docs and launch content
3	Multi-tenancy is deferred from v1.0 (MSPs run one instance per client); full remediation for one provider, second provider experimental	Months 4, 5 and 7 were overloaded; the old plan already said to cut tenancy if needed
4	Injection starter tests, privacy basics (retention, log redaction, delete-case) and the Microsoft 365 go/no-go move to Month 2	v0.1 goes public in Month 2 and handles hostile, personal data before the old safeguards existed
5	Buffer and integration week at the end of Months 3 and 6, plus a pre-agreed cut list	All 32 weeks were allocated; delays had nowhere to go
6	Evaluation protocol: frozen test set by end of Month 2, numeric targets set after the baseline, fresh final test set in Month 7	Early synthetic results look too good and are easy for a panel to challenge
7	Written tie-break and ownership agreement before the first public commit	Two equal owners can deadlock

#	Change	Reason
8	One-sentence research contribution and monthly report chapters	Final-year panels ask what is new; writing everything in Month 8 is risky
9	Small fixes: visible demo in Month 2, neutral sandbox egress, minimum dataset size and scripted synthetic data	See sections 6, 7 and 15

2. Scope of v1.0

PhishDesk is an open-source, self-hosted phishing triage and response platform. An employee reports a suspicious email; the platform analyses it, explains a Safe / Suspicious / Dangerous verdict with reasons and uncertainty, finds related messages, and can quarantine them only after human approval.

In v1.0: IMAP and Gmail connectors (Microsoft Graph at least experimental), defensive parser, security checks, isolated sandbox, AI classification and injection-hardened LLM analysis, brand matching, campaign clustering, approval-gated quarantine and restore for one provider, dashboard, plugin API, benchmark.

Deferred or experimental: multi-tenancy (future work; documented one-instance-per-client setup instead), second remediation provider, any stretch experiment.

Out of scope: inbound mail gateway, real-time filtering, antivirus engine, awareness training.

Research contribution (one sentence, refine by Month 2): measuring how much a prompt-injection-hardened LLM layer, grounded in deterministic evidence, improves phishing triage over rules-only and classifier-only baselines, and publishing a reusable injection test suite.

3. Ownership and decisions

	Person 1: Cybersecurity and backend	Person 2: AI/ML and dashboard
Owns	Connectors (IMAP, Gmail, Graph), MIME parser, SPF/DKIM/DMARC evidence, check plugins, sandbox, backend API and authorisation, remediation, Docker and deployment, security review, security CI	Datasets and labels, baseline classifier, local LLM analysis, injection defence, decision fusion, brand matching, embeddings, clustering, calibration, benchmark harness and ML CI, **React dashboard**, explanations
Hands over	OpenAPI spec and a mock API by Week 5 so the dashboard can be built in parallel	Fixtures, model cards, benchmark results
Shared	Contracts, code review, documentation, releases, community work	

Tie-break rule. Each owner has the final say inside their own track. Cross-track decisions need both. If you deadlock: write down the options and a measurable success criterion, run a time-boxed spike (maximum one week), and pick the option that meets the criterion. If that fails, ask your supervisor or an agreed neutral mentor.

Written agreement (finish before the first public commit; check your university's rules on student-project IP first). Cover: IP ownership and revenue or equity split; licence (Apache-2.0 is the working choice); contribution policy (DCO is simpler; a CLA keeps dual-licensing open); authorship order on reports and papers; who holds the GitHub organisation, domain and package names (both with 2FA, recovery codes stored safely); what happens if one of you leaves; and how commercial offers are decided.

4. Shared contracts (agree in Week 1)

Contract	Direction	Contents
Evidence v1	Cyber to AI	Parsed case data, sender and authentication signals, links, attachments, sandbox artefacts, findings, missing-evidence states

Contract	Direction	Contents
Verdict v1	AI to API	Case ID, Safe / Suspicious / Dangerous label, confidence, evidence-grounded reasons, model signals, injection flags
ActionRequest v1	Campaign to Cyber	Requested action and scope, reason, risk, approval requirement, rollback description; execution stays with the authorised backend

All three are versioned Pydantic models with JSON Schema and fixtures. Model output is untrusted until validated. An LLM can never authorise quarantine or weaken deterministic safeguards, and a failed lookup is never treated as proof of safety.

5. Milestones at a glance

Month	Version	Theme	Community task	Gate at month end
1	v0.0	Foundation	Name check, licence decision, CoC, SECURITY, CONTRIBUTING stub	Sandbox spike and local LLM go/no-go
2	v0.1	First verdict (public soft launch)	README with demo, 15-20 good first issues	Microsoft 365 tenant go/no-go; test set frozen
3	v0.2	Sandbox and dashboard	Docs site skeleton, first plugin example	Containment tests pass or no live URLs
4	v0.3	Brands and campaigns	Brand-template guide, demo video	Mid-project demo
5	v0.4	Approved response	Publish benchmark v1	One-provider quarantine and restore works
6	v0.5	Hardening	Plugin tutorial, docs site live	Plugin API frozen
7	v0.9	Beta	Beta announcement, injection post draft	Outside tester succeeds

Month	Version	Theme	Community task	Gate at month end
8	v1.0	Launch	Launch posts, awesome-list PRs, talk submissions	Fresh reviewer completes the full demo

6. Month 1 (Weeks 1-4): v0.0 Foundation

Goal: a repeatable development environment where both tracks can be tested independently through the contracts.

Person 1

- FastAPI project layout, linting, type checks, pytest, logging, CI, Docker Compose skeleton.
- Defensive parser for synthetic .eml files (headers, bodies, URLs, QR codes, attachment metadata) with size, recursion and time limits.
- First checks: reply-to mismatch, lookalike domain, suspicious URL structure, domain age; keep authentication context accurate.
- Mail access spike: IMAP, Gmail and Microsoft 365 test accounts; **request or set up a Microsoft 365 test tenant now**.
- Sandbox feasibility: disposable browser screenshot on an isolated VM with private networks and credentials blocked.

Person 2

- Dataset plan and labels (credential, payment, urgency, impersonation, legitimate, legitimate-but-urgent) with provenance. **Set a minimum size** (at least several hundred labelled emails, with a meaningful share of legitimate-but-urgent) and write a script that generates synthetic emails with invented brands and lab-only domains.
- Reproducible baseline (TF-IDF plus logistic regression): precision, recall, F1, confusion matrix.
- Local LLM feasibility on about 20 synthetic emails: validity, latency, quality, memory. Record the model and hardware.
- Fake analyser returning validated fixtures.

Community and academic: sign the written agreement draft; check the name for conflicts; add licence, CODE_OF_CONDUCT, SECURITY and CONTRIBUTING stubs; write report chapter 1 (problem, market gap, related work).

Weeks: W1 repo, CI, draft contracts and agreement. W2 parser on 20 samples, baseline, LLM speed test, sandbox spike. W3 IMAP prototype, domain checks, LLM intent prototype, first injection examples. W4 connect parser and checks to the mock AI and the model to mock Evidence; run a synthetic end-to-end prototype; 30-minute retrospective.

Acceptance: Cyber and AI exchange schema-valid Evidence and Verdict; tests pass; blockers recorded. **Gate:** if the local LLM is too slow on your hardware, plan a rules-first fast path and a smaller model before Month 2.

Safety: lab and synthetic data only; no live remediation; no uncontrolled external navigation.

7. Month 2 (Weeks 5-8): v0.1 First verdict

Goal: report, analyse, explain and reply, safely, and go public.

Person 1

- IMAP and Gmail connectors: secure credentials, deduplication, provider errors, retries, mock connectors.
- Standardised check pipeline (check, severity, confidence, evidence; unknown stays unknown).
- Redis plus Dramatiq or Celery, PostgreSQL case records, job states, idempotent retries.
- Case API with a published OpenAPI spec and mock API by Week 5; sanitised plain-language reply with no clickable dangerous links.
- **Privacy basics now:** configurable retention, log redaction, delete-case endpoint.
- Clean Docker Compose install. **End-of-month gate:** can you obtain Microsoft 365 test-tenant access? If not, Graph becomes experimental or deferred and IMAP plus Gmail remain the core.

Person 2

- Structured LLM intent extraction (credential and payment requests, urgency, impersonation) grounded in message evidence.
- Decision fusion: deterministic high-risk evidence sets a minimum risk the model cannot override.
- Explanations with uncertainty and missing-evidence handling.
- **Starter injection set (10-20 cases) running as CI regression tests.**

- **Freeze the test set** (hashed, never tuned on) and keep a separate development set; record baseline numbers.
- A visible demo surface for the README: a minimal case page or clean CLI output; scaffold the React app against the mock API.

Community and academic: README with 60-second demo and quick start; 15-20 well-written `good first issue` and `help wanted` tickets; enable Discussions; commit to 48-hour replies; make the repo public (soft launch). Report chapter 2 (requirements, data and method).

Weeks: W5 contract integration, API spec. W6 async pipeline demo. W7 case API, reporter reply, review of mismatches. W8 Docker quick start, integration tests, v0.1 release.

Acceptance: a lab email is reported, parsed, classified, explained and answered; measure verdict time against the one-minute target on named hardware with a named model (fast path allowed). **Numeric targets** for false alarms and injection success are set now from the baseline, and written into the repo.

Safety: sanitise attacker content in replies; never treat a failed lookup as safe.

8. Month 3 (Weeks 9-12): v0.2 Sandbox and dashboard

Goal: inspect linked pages in isolation and give analysts a secure evidence dashboard. Week 12 is a buffer and integration week.

Person 1

- Disposable Playwright / Chromium jobs: screenshots, redirects, final URL, resource indicators, form fields.
- Containment: dedicated VM or equivalent, egress proxy, blocked private and metadata ranges, DNS and redirect checks, non-root, quotas.
- **Neutral egress:** visits use a neutral address and strip identifying tokens where feasible, because a visit can confirm an active target to the attacker; add an admin setting to disable link visiting and record this in the threat model.
- Evidence API with screenshot IDs, failure and timeout states; authenticated case endpoints.

Person 2

- Injection guard: separate hostile email from instructions, validate output, independent check for instruction-following.

- Full injection suite: direct, invisible, encoded, multilingual, attachment and image-carried cases, with measured success rates.
- Grounded page analysis (form and redirect evidence, no invented observations).
- **Dashboard v1:** case list, evidence view, screenshots, reasons, uncertainty, injection flags.

Community and academic: MkDocs site skeleton; one worked example plugin (a simple check); report chapter 3 (architecture, threat model).

Weeks: W9 isolated service, injection threat model. W10 redirects and screenshots, input separation. W11 SSRF and form tests, adversarial regression. W12 buffer, integration, v0.2 release.

Acceptance: a synthetic linked-page case gives safe screenshot and form evidence and a validated verdict; mandatory containment tests pass. **Gate:** until containment is proven, no unknown live URLs.

Safety: a container alone is not a sandbox guarantee.

9. Month 4 (Weeks 13-16): v0.3 Brands and campaigns

Goal: find counterfeit sign-in pages and group related emails, including unreported copies in authorised lab mailboxes.

Person 1

- Microsoft Graph connector (read and search, scopes, consent, throttling, retries) in the test tenant; experimental if access was limited.
- Infrastructure features and graph edges: domains, redirect destinations, IPs, Reply-To identities, attachment hashes.
- Scoped `search_copies()` with pagination, permissions and rate limits; campaign API.

Person 2

- Brand corpus: approved reference login screenshots, expected domains, provenance. Start with a small number of brands.
- Visual matching (OpenCLIP or ONNX embeddings), compared with domain legitimacy.
- Versioned text and URL embeddings in pgvector; clustering that combines embeddings with shared infrastructure and explains membership.
- Evaluation: brand detection, cluster quality, false associations, robustness to content variation.

- Dashboard campaign view.

Community and academic: brand-template contribution guide (data-only contributions); record a demo video; report chapter 4 (detection methods).

Weeks: W13 permissions, brand library and baseline matcher. W14 graph extraction, screenshot embeddings. W15 mailbox search, clustering. W16 connector tests, tuning, **mid-project v0.3 demo**.

Acceptance: synthetic related messages form a defensible cluster and related copies are found only in authorised mailboxes.

Safety: a lookalike screenshot alone is not proof of fraud; require independent corroboration.

10. Month 5 (Weeks 17-20): v0.4 Approval-gated response

Goal: human-approved quarantine and restore, calibrated confidence, benchmark v1.

Person 1

- ActionRequest scoping, policy checks, dry run, execution disabled by default.
- Role-based approvals, proposal expiry, scope revalidation right before action, rate limits, idempotency.
- **Quarantine and restore for one provider, fully tested** (Graph or Gmail, whichever works best in your lab); the other is marked experimental.
- Audit log: proposer, approver, targets, timestamps, outcome, restoration, with redaction and access checks.

Person 2

- Confidence calibration (reliability plots, Brier score, by case category and missing evidence).
- Advisory triage priority from fused evidence; never an authorisation.
- Benchmark v1: phishing and false alarms, explanations, brands and campaigns, injections, runtime and cost; failure analysis.
- Dashboard approval queue, scope preview, execution results and restore controls.

Community and academic: publish benchmark v1 and its README table; report chapter 5 (remediation design, calibration).

Weeks: W17 policies, dry run, calibration labels. W18 quarantine, RBAC, calibrated fusion. W19 restore, audit, scope checks, benchmark. W20 approval UI, adversarial tests, v0.4 release.

Acceptance: synthetic campaign, find copies, propose, approve, quarantine, audit, restore; incorrect removals in the test set are zero.

Safety: no quarantine solely because of a model label.

11. Month 6 (Weeks 21-24): v0.5 Hardening

Goal: review security, freeze documented plugin interfaces, produce defensible comparisons. Week 24 is a buffer and integration week.

Person 1

- Threat model and testing: API permissions, mailbox scoping, parser fuzzing, secrets, sandbox escape, SSRF, remediation bypass.
- Versioned, documented plugin API (checks, connectors, analysers, brand templates, actions) with permissions and fixtures, plus sample plugins.
- CI safety: CodeQL, dependency and secret scanning, image scans, type and lint checks.
- Deployment docs: credential permissions, backup and restore, upgrades. If time remains, work on the second provider's remediation (experimental).

Person 2

- Ablations on the same data: rules only, classifier only, classifier plus LLM, full pipeline.
- Model comparison (local and cloud): quality, valid-JSON rate, hallucinations, runtime, memory, cost.
- Robustness: time-separated samples, missing evidence, formats, languages, new synthetic attack variants.
- Explanation feedback from about ten volunteers if feasible; pinned configurations and recorded dataset sources and limits.

Community and academic: plugin tutorial (add a check in 30 minutes); docs site live; report chapter 6 (evaluation draft).

Weeks: W21 threat model, ablation harness, evaluation snapshot. W22 security tests, model comparison. W23 plugin API, CI policy, robustness. W24 buffer, docs, v0.5 release, **plugin API frozen**.

Acceptance: mandatory security and contract tests pass; benchmark claims are reproducible.

Safety: permission declarations do not isolate hostile plugin code; review every executable contribution.

12. Month 7 (Weeks 25-28): v0.9 Public beta

Goal: outside testers can install and evaluate PhishDesk. Multi-tenancy is no longer in scope.

Person 1

- Reliability: worker failure recovery, retries, backups, upgrades, retention.
- MSP guide for running one instance per client; short design note on future multi-tenancy as future work.
- Beta operations: clean-start install guide, release notes, private vulnerability reporting, tester onboarding.
- Fix permission, data-segregation and reliability defects before anything optional.

Person 2

- **Build the fresh final test set** (new templates, brands, languages) that nobody tuned on.
- Final stress tests: injections, brand matcher, clusters, uncertainty under changing templates.
- Stretch experiment **only if the cut list allows it** (recommended: lab-generated AI-written phishing robustness with fake brands).
- Tester feedback analysis; inference tuning (cache, batching, model size) without bypassing security evidence; dashboard polish.

Community and academic: beta announcement; tester onboarding; draft the blog post on defending an LLM email analyst from prompt injection; report chapter 7 (results and limitations).

Weeks: W25 beta readiness check, final test set. W26 reliability and robustness work. W27 open beta to testers. W28 fix blockers, v0.9 readiness review.

Acceptance: an outside tester installs, runs a synthetic demonstration and reports a reproducible issue; no unsupported claims.

13. Month 8 (Weeks 29-32): v1.0 Launch

Goal: a verified public release with clear docs, reproducible results and a complete demo.

Person 1

- Security regression: authentication, secrets, mailbox scope, parser abuse, sandbox egress, approvals, audit, restore.
- Release engineering: tag, changelog, supported deployment matrix, Compose setup, migration notes.
- Operations docs: provider permissions, backup, retention, privacy, security and responsible use.
- Repeatable demo lab with owned test accounts; fix serious issues or remove the related claims.

Person 2

- Final benchmark: precision and recall, calibration, injection success by category, brand and campaign quality, latency and resources, on the fresh test set.
- Methods report: licences, splits, ablations, model comparisons, limitations; model documentation and known failures.
- Final presentation: AI assists analysts, while verified findings and human approval control sensitive actions.

Community and academic: publish launch posts a few days after the tag, once docs are verified; submit pull requests to relevant awesome lists; submit talks to local security and university events; finalise the report and abstract.

Weeks: W29 release candidate, freeze evaluation data. W30 end-to-end regression, benchmark draft. W31 docs, demo rehearsal. W32 tag, security sign-off, launch.

Acceptance: a fresh reviewer installs, evaluates a synthetic message, sees the evidence, finds copies, then safely approves, quarantines and restores.

Safety: do not promise perfect detection, immunity to injection or error-free remediation.

14. Cut list (agree now)

If you are more than a week behind at a monthly gate, cut from the top. Never cut the last group.

1. Stretch experiment

2. Second remediation provider (stays experimental)
3. Campaign explanation polish
4. Microsoft Graph connector (experimental or deferred)
5. Number of brands in the corpus
6. Dashboard polish

Never cut: sandbox containment, approval controls, privacy and redaction, contracts and tests, honest benchmark reporting.

15. Evaluation protocol

- **Frozen test set** by the end of Month 2: hashed, committed, never tuned on. A separate development set is used for tuning.
- **Time-separated splits** wherever possible; include legitimate-but-urgent emails from the start.
- **Targets** (false-alarm rate, injection success rate, calibration error) are set after the Month 2 baseline and recorded in the repo; do not invent them earlier.
- **Fresh final test set** built in Month 7 with new templates, brands and languages.
- **Latency:** name the model and hardware; keep a fast path for obvious cases. The one-minute goal is measured on that setup.
- **Comparisons:** rules only, classifier only, classifier plus LLM, full pipeline, all on the same data; injection attack success reported by category before and after safeguards.
- **Installation:** measure clean Docker Compose setup to first success against the ten-minute goal.
- **Data policy:** synthetic emails with invented brands and lab-only domains, or appropriately licensed public data. Never commit real employees' emails, tokens or secrets. Mailbox actions only in authorised test accounts.

16. Working agreement

- **GitHub:** a jointly owned organisation with both as Owners; this is not an IP agreement. `main` stays stable; short-lived `feature/cyber-`, `feature/ai-`, `feature/contracts-`, `fix/` and `docs/` branches; squash merges.
- **Reviews:** every PR is reviewed by the other owner and needs passing CI. CODEOWNERS reflects the ownership table in section 3.

- **Issues:** one lead, one reviewer, source package, dependencies, tasks, test cases, definition of done.
- **Cadence:** Monday 30-minute planning; Friday 30-minute runnable demo; fortnightly integration demo; monthly release gate against the table in section 5.
- **Capacity:** plan at about 70 percent of theoretical time, with the two buffer weeks protected.
- **Report:** one chapter per month (section 5 lists the Month 1-7 chapters; Month 8 finalises).

17. Top risks

Risk	Mitigation
No Microsoft 365 test tenant	Go/no-go at end of Month 2; Graph experimental; IMAP and Gmail core
Local LLM too slow or weak	Month 1 test; smaller model; rules-first fast path; optional cloud model
Sandbox not provably contained	No live URLs until tests pass; dedicated VM; disable link visiting option
Dataset quality or licence problems	Scripted synthetic data plus licensed public data; provenance documented
Scope creep	Cut list; stretch work only after v0.5
One owner unavailable	Cross-review; both understand the full pipeline; written leave scenario
Low adoption	README and demo in Month 2, good first issues, benchmark and blog content, no purchased stars
Name or IP conflict	Check in Month 1; written agreement before going public

18. Definition of done (every change)

- Code works locally and in an integration fixture; inputs and outputs are schema-validated.

- Unit, security and regression tests run in CI.
- Documentation and examples updated; the other owner reviewed the PR.
- No secrets or private emails committed; failures and unknown evidence are handled safely.
- Benchmarks do not silently regress; every release claim is backed by a measurement.
- Optional features are cut before privacy, containment or approval controls.